

Task 1 Agent :

Move selection approach :

The agent selects a move by generating all legal moves from the current position, evaluating the resulting board for each candidate, and choosing the move with the highest evaluation for the player to move.

The agent works as follow :

- Generate all legal moves for the current board
- Shuffle the move list
- For each move :
 - Make the move on a copy of the board
 - Evaluate the resulting board state using a fast, single step evaluation function
- Pick the move with the best score according to that evaluation

Since the evaluation is done after one depth only (agent->opponent->evaluation), the agent is thus using 1-depth search

Algorithm/Logic :

- Move generation :
 - The agent obtains the raw legal move matrices from `chess_obj.legal_moves()` and then converts them into explicit move tuples using `piece_matrix_to_legal_moves(...)`. Each move is represented as `((i, j), (dx, dy), promo)`, where `(i, j)` is the source square, `(dx, dy)` is the destination offset, and `promo` specifies a promotion piece if applicable.
- Candidate Evaluation Loop :
 - The agent randomizes the move order with `random.shuffle(moves)` so the behaviour is not deterministic on moves with equal scores.
 - For each candidate move :
 - The agent copies the board
 - It plays the move on the copy
 - It calls `score = self.feval(s1, root_turn)` to evaluate the resulting position from the perspective of the original player
 - Choosing the best move :
 - The agent keeps the currently best move and best score, updating when a candidate yields a strictly better score
 - After trying all candidates, it returns the best move found

Evaluation Functions :

- **feval(s1, root_turn) :**

Python

```
def feval(self, s1, root_turn):  
    pm2, pr2 = s1.legal_moves()  
    opp_moves = piece_matrix_to_legal_moves(pm2, pr2)  
    if not opp_moves: return self.deval(s1, root_turn)
```

```

threatened=self.nogo_squares(s1)
score=self.meval(s1,root_turn)
for i in range(5):
    for j in range(4):
        t,c=s1.any_piece_at(i,j)
        if t!=-1 and c==root_turn and (i,j)in threatened:
            score-=self.piece_values[t]*0.8
return score

```

- Generate opponent legal moves in s1 (the position after the candidate move).
- If the opponent has no legal moves (checkmate/stalemate), fallback to deval(s1, root_turn) to get a correct terminal evaluation.
- Else:
 - Compute threatened = nogo_squares(s1)
 - Get a quick material-only evaluation score = meval(s1, root_turn)
 - Penalize friendly pieces that currently sit on squares threatened by the opponent (i.e., pieces that can be captured next move) by subtracting piece_value * 0.8 for each such piece.
 - Return the modified score
- **deval(s1, root_turn) :**

Python

```

def deval(self, state:Chess, root_turn:int):
    if not state.has_legal_moves:
        r=state.game_result()
        if(r==1 and root_turn==1)or(r==-1 and root_turn==0):return 99999
        elif(r==-1 and root_turn==1)or(r==1 and root_turn==0):return -99999
        return 0
    score=0
    for i in range(5):
        for j in range(4):
            t,c=state.any_piece_at(i,j)
            if t!=-1:
                v=self.piece_values[t]
                if c==1:
                    score+=v
                if t==0:
                    if i==4:score+=500
                    elif i==3:score+=200
                    else:score+=i*30
            else:
                score-=v
                if t==0:

```

```

        if i==0:score-=500
        elif i==1:score-=200
        else:score-=(4-i)*30
    return score if root_turn==1 else -score

```

- If state.has_legal_moves is False, call state.game_result() :
 - If game_result() indicates the root player won, return a very large positive value (99999)
 - If the root player lost, return a very large negative value (-99999)
 - Otherwise return 0 (draw)
- If the game is not terminated :
 - Compute a fuller position score by iterating the board and :
 - Adding/subtracting piece material values (self.piece_values[t])
 - For pawns (type t == 0) add extra bonus for advanced pawns (depends on rank i)
- Return score with sign depending on root_turn
- **meval(state, root_turn) :**

Python

```

def meval(self, state:Chess, root_turn:int):
    s=0
    for i in range(5):
        for j in range(4):
            t,c=state.any_piece_at(i,j)
            if t!=-1:
                v=self.piece_values[t]
                s+=v if c==1 else -v
    return s if root_turn==1 else -s

```

- Sum piece values for white and subtract for black (or vice-versa depending on root_turn)
- Return s or -s depending on which side is the root player

Heuristics, evaluation terms and rules :

- Material Values (self.piece_values) :
 - Each piece type has a fixed value
 - These values are used by both meval() and deval()
- Pawn advancement bonus :
 - Pawns are rewarded for advancing toward promotion :
 - If a pawn belongs to player 1 and sits nearer the promotion rank, deval adds extra points (specific +500 or +200 for edge ranks or i*30 for intermediate ranks)
 - Symmetric negative adjustments are done for the opponent pawns
- Terminal detection and extreme game scores :

- If the position is checkmate for the root player, return +99999; if checkmate against the root player, return -99999. Draws return 0
- Threat penalty :
 - After a candidate move, if any of the root player's pieces are sitting on squares the opponent can capture next move (determined via `nogo_squares`), subtract $0.8 * \text{piece_value}$ from the score for that piece

Output (as per the autograder) :

```
Playing Task1Agent vs RandomAgent.: 100% | 100/100 [00:06<00:00, 15.46it/s]
=== FINAL RESULTS ===
Task1Agent:
  Wins as White: 32
  Wins as Black: 29
  Total Wins:    61
  Avg Time for each move: 0.003359 sec
RandomAgent:
  Wins as White: 0
  Wins as Black: 0
  Total Wins:    0
  Avg Time for each move: 0.000370 sec
Draws: 39
Avg Game Length (full moves): 13.0 full moves
Avg Game Length (half moves): 27.3 plies
TASK-1 score: 61
PASSED
```

Task 2 Agent :

Move selection approach :

The agent's move selection process is initiated by the move method. This function acts as the "root" of the search, setting up and launching the main algorithm.

- **Getting Legal Moves** : The code first calls `chess_obj.legal_moves()` and `piece_matrix_to_legal_moves()` to get a list of all possible moves from the current position.
 - If there are no legal moves, it returns None
 - If there is only one legal move, it returns that move immediately without any searching
- **Order Moves** :
 - Before starting the search, the agent sorts all legal moves using the `self.omf` (Order Moves Function) heuristic. This function prioritizes moves that are likely to be good (like captures and promotions) to improve search efficiency.
- **Iterate and Search** :
 - The agent then loops through this ordered list of moves. For each move :
 - It creates a `board_copy` of the current game state

- It makes the move on that copy
 - It calls the core self.minimax algorithm on this new board state. This call is for a depth of MAX_DEPTH - 1
 - The maximizing flag is set to (board_copy.turn == 1), which means it's now the opponent's turn to move, and the minimax function will be evaluating from their perspective
- **Select best move :**
 - The move function initializes a best_score
 - As it gets a score back from each minimax call, it compares it to the best_score
 - If the current move's score is better, it updates best_score and sets best_move to the current move
- **Result :** After checking all root-level moves, the function returns the best_move it found

Algorithm/Logic :

The core logic the agent uses is the Minimax algorithm, enhanced with Alpha-Beta Pruning. This logic is contained entirely within the minimax function. It is a recursive function that explores the game tree of possible future moves to find fixed depth of 3 (MAX_DEPTH = 3)

- **Maximizing vs Minimizing :** When the maximizing parameter is True, the function represents the "Maximizing" player and tries to find the move that leads to the highest score. When False, it represents the "Minimizing" player and tries to find the move that leads to the lowest score
- **Base case :** The recursion stops in these 2 cases :
 - Terminal Node :
 - Agent wins : Returns 100000 score
 - Opponent wins : Returns -100000 score
 - Draw : Returns 0
 - Depth Limit Reached : When the search has gone as deep as MAX_DEPTH, it stops recursing. It cannot search any further, so it calls self.eval(chess_obj) to get a static "snapshot" score of the current board position. This score is its best guess for how good the position is
- **Recursive Step :**
 - Get and Order Moves : This gets all the legal moves for the current board and sorts the using self.omf(), heuristics ordering prior to search
 - Loop over ordered Moves :
 - Copy board (board_copy = chess_obj.copy()) and apdepth make_move
 - Call minimax on the resulting position with depth = MAX_DEPTH - 1 and maximizing = (board_copy.turn == 1)
 - Update best_move using the returned score
 - Minimax Implementation using alpha-beta pruning :
 - If Maximizing :
 - It loops through the ordered moves, making each one on a new_board
 - It recursively calls self.minimax with depth - 1 and flips the maximizing flag to False
 - It updates alpha (the best score the maximizer has found so far

- It checks if $\beta \leq \alpha$: break. This is the alpha-beta cutoff. It means the minimizer (β) has already found a move in another part of the tree that is worse for the maximizer than the best move the maximizer (α) has already secured. There is no point in searching this branch further, so it "prunes" it
- If Minimizing :
 - It loops through moves, recursively calls minimax (flipping maximizing to True), and updates β (the best score the minimizer has found)
 - It also checks if $\beta \leq \alpha$: break. It means the maximizer (α) has found a move that is better than what the minimizer (β) is currently guaranteed. The maximizer will never choose this path, so the minimizer stops searching

Heuristics, evaluation terms and rules :

- Material Values :
 - These are the base material weights. Pawn=100, Knight=300, Bishop=350 etc.

Python

```
PIECE_VALUE = {
    0: 100,    # Pawn
    1: 300,    # Knight
    2: 350,    # Bishop
    3: 500,    # Rook
    4: 900,    # Queen
    5: 9000    # King (very large)
}
```

- Piece Square tables :
 - Each piece type has a 5×4 table
 - eval() adds/subtracts the value from these tables for positional bonus :
 - For white pieces (color == 1) it uses table[r][c]
 - For black pieces it uses table[4 - r][c] and subtracts it from score
 - The final evaluation : material + piece_square_bonus

Python

```
PAWN_TABLE = [
    [0, 0, 0, 0],
    [5, 10, 10, 5],
    [10, 20, 20, 10],
    [20, 40, 40, 20],
    [0, 0, 0, 0]
]
KNIGHT_TABLE = [
```

```

        [-10, 0, 0, -10],
        [0, 10, 10, 0],
        [5, 20, 20, 5],
        [5, 15, 15, 5],
        [-10, 0, 0, -10]
    ]
    BISHOP_TABLE = [
        [0, 5, 5, 0],
        [5, 10, 10, 5],
        [5, 15, 15, 5],
        [5, 10, 10, 5],
        [0, 5, 5, 0]
    ]
    ROOK_TABLE = [
        [0, 5, 5, 0],
        [10, 15, 15, 10],
        [5, 10, 10, 5],
        [5, 10, 10, 5],
        [0, 5, 5, 0]
    ]
    QUEEN_TABLE = [
        [0, 5, 5, 0],
        [5, 10, 10, 5],
        [5, 15, 15, 5],
        [5, 10, 10, 5],
        [0, 5, 5, 0]
    ]
    KING_TABLE = [
        [20, 30, 30, 20],
        [10, 15, 15, 10],
        [0, 0, 0, 0],
        [-10, -10, -10, -10],
        [-20, -30, -30, -20]
    ]

    PIECE_SQUARE_TABLES = {
        0: PAWN_TABLE,
        1: KNIGHT_TABLE,
        2: BISHOP_TABLE,
        3: ROOK_TABLE,
        4: QUEEN_TABLE,
        5: KING_TABLE
    }
}

```

- Move ordering :

- `omf(moves, chess_obj)` :
 - For each move, it computes a small "score" :
 - If the move captures (dest is a piece): `score += PIECE_VALUE[dest[0]] * 10`
 - If the move is a promotion: `score += 5000`
 - Return moves sorted by descending score
 - This captures/promotions to the front
- `ocof(moves, chess_obj)` :
 - Similar to `omf()`, but only returns captures and promotions
 - Score weighting identical: captures get `PIECE_VALUE[victim]*10`, promotions get `+5000`
- Terminal Scoring : `game_result()` maps to ± 100000 or 0 for draw.
- Evaluation :
 - Loops `r` from 0..4 and `c` from 0..3
 - `piece = chess_obj.any_piece_at(r, c)`, returns a tuple (`piece_type`, `color`) or a tuple with first element -1 when empty
 - For a piece: `val = PIECE_VALUE[piece_type]`, `table = PIECE_SQUARE_TABLES[piece_type]`
 - If piece is white (`color == 1`) :
 - `material += val`
 - `piece_square_bonus += table[r][c]`
 - If black :
 - `material -= val`
 - `piece_square_bonus -= table[4 - r][c]`
 - Return `material + piece_square_bonus`

Output (as per the autograder) :

```

Playing Task2Agent vs RationalAgent.: 100%|██████████| 100/100 [04:16<00:00, 2.56s/it]
=== FINAL RESULTS ===
Task2Agent:
  Wins as White: 26
  Wins as Black: 28
  Total Wins:    54
  Avg Time for each move: 0.048707 sec
RationalAgent:
  Wins as White: 1
  Wins as Black: 2
  Total Wins:    3
  Avg Time for each move: 0.108065 sec
Draws: 43
Avg Game Length (full moves): 16.0 full moves
Avg Game Length (half moves): 32.6 plies
TASK-2 score: 51
PASSED

```

Task 3 Agent :

General Idea :

The agent uses a Minimax search with alpha–beta pruning to a fixed search depth (MAX_DEPTH = 3). At leaf nodes it uses a static evaluation that combines material values and piece-square table bonuses. To reduce horizon effects, it performs a shallow quiescence search (depth q_depth = 1) which extends search only along capture and promotion moves. Move ordering heuristics (captures weighted by captured piece value $\times 10$, promotions given a large bonus) are applied to improve alpha–beta effectiveness. If no best move is found (edge case), the agent picks a random legal move.

Move generation and selection approach :

- Legal moves are generated using the fastchess api calls
- Each move has the structure: ((i,j), (dx,dy), promotion), where :
 - (i,j) : origin square (row, column)
 - (dx,dy) : delta to destination (so destination = (i+dx, j+dy))
 - promotion : integer indicating promotion piece (or -1 / falsy if no promotion)
- Move Logic :
 - Get legal moves and convert to a list
 - If there are no legal moves $\rightarrow$ return None
 - If exactly one legal move $\rightarrow$ return it immediately (forced move)
 - Initialize best_move and best_score depending on side to move
 - Order moves with order_moves() (captures/promotions priority)
 - For each ordered move :
 - Make a copy of the board
 - Play the move on the copy: board_copy.make_move(...)
 - Call recursive minimax() on MAX_DEPTH - 1
 - Update best_score/best_move according to whether we are maximizing or minimizing
 - Pick a random legal move as fallback, if no best_move is selected

Algorithm/Logic :

Minimax with Alpha-Beta Pruning :

- Base case :
 - If result is terminal :
 - result == 1 $\rightarrow$ return 100000
 - result == -1 $\rightarrow$ return -100000
 - Otherwise (draw) $\rightarrow$ return -300
 - If depth == 0 $\rightarrow$ call qsearch() to avoid evaluating “noisy” tactical positions
- Recursive Step :
 - Generate moves (legal moves) and moves = order_moves(moves, chess_obj)
 - If maximizing :
 - Initialize max_eval = -10^{**9}
 - For each move :
 - simulate new_board

- `eval_score = self.minimax(new_board, depth-1, alpha, beta, False)`
 - `max_eval = max(max_eval, eval_score)`
 - `alpha = max(alpha, eval_score)`
 - If `beta <= alpha` → break
- return `max_eval`
- If minimizing :
 - Initialize `min_eval = +10**9`
 - For each move :
 - simulate `new_board`
 - `eval_score = self.minimax(new_board, depth-1, alpha, beta, True)`
 - `min_eval = min(min_eval, eval_score)`
 - `beta = min(beta, eval_score)`
 - If `beta <= alpha` → break
 - return `min_eval`

Quiescence search : Static evaluation sometimes can mis-evaluate positions where a capture is imminent, this search extends leaf evaluation down capture/ promotions so evaluation is done in a stable position

- Called when `depth == 0` in minimax
- `stand_pat_score = self.evaluate(chess_obj)` : baseline (non-moving) score
- If `q_depth == 0` -> return `stand_pat_score`
- Update `alpha/beta` with `stand_pat_score` and check cutoff if `beta <= alpha`
- Collects capture and promotion moves only via `order_captures_only(moves, chess_obj)`:
 - If no capture/promotion moves -> return `stand_pat_score`
- For each capture/promotion :
 - simulate and recursively call `qsearch(new_board, alpha, beta, not maximizing, q_depth-1)`
 - update `alpha` or `beta`
 - check `alpha-beta` cutoff

Heuristics, evaluation terms and rules :

- Material Values :
 - These are the base material weights. Pawn=100, Knight=300, Bishop=350 etc.

Python

```
PIECE_VALUE = {
    0: 100,    # Pawn
    1: 300,    # Knight
    2: 350,    # Bishop
    3: 500,    # Rook
    4: 900,    # Queen
    5: 9000    # King (very large)
}
```

- Piece Square tables :

- Each piece type has a 5×4 table
- eval() adds/subtracts the value from these tables for positional bonus :
 - For white pieces (color == 1) it uses table[r][c]
 - For black pieces it uses table[4 - r][c] and subtracts it from score
 - The final evaluation : material + piece_square_bonus

Python

```
PAWN_TABLE = [
    [0, 0, 0, 0],
    [5, 10, 10, 5],
    [10, 20, 20, 10],
    [20, 40, 40, 20],
    [0, 0, 0, 0]
]
KNIGHT_TABLE = [
    [-10, 0, 0, -10],
    [0, 10, 10, 0],
    [5, 20, 20, 5],
    [5, 15, 15, 5],
    [-10, 0, 0, -10]
]
BISHOP_TABLE = [
    [0, 5, 5, 0],
    [5, 10, 10, 5],
    [5, 15, 15, 5],
    [5, 10, 10, 5],
    [0, 5, 5, 0]
]
ROOK_TABLE = [
    [0, 5, 5, 0],
    [10, 15, 15, 10],
    [5, 10, 10, 5],
    [5, 10, 10, 5],
    [0, 5, 5, 0]
]
QUEEN_TABLE = [
    [0, 5, 5, 0],
    [5, 10, 10, 5],
    [5, 15, 15, 5],
    [5, 10, 10, 5],
    [0, 5, 5, 0]
]
KING_TABLE = [
    [20, 30, 30, 20],
    [10, 15, 15, 10],
    [0, 0, 0, 0],
```

```

        [-10, -10, -10, -10],
        [-20, -30, -30, -20]
    ]
    PIECE_SQUARE_TABLES = {
        0: PAWN_TABLE,
        1: KNIGHT_TABLE,
        2: BISHOP_TABLE,
        3: ROOK_TABLE,
        4: QUEEN_TABLE,
        5: KING_TABLE
    }

```

- Move Ordering Heuristics :
 - **Captures first** : capture moves get a score $\text{PIECE_VALUE}[\text{captured_type}] * 10$. Multiplying by 10 boosts captures over quiet moves ensuring captures are considered earlier
 - **Promotions high** : promotions are given a large bonus +5000 to strongly prioritize promotion moves
 - **Sorting descending** : moves sorted by -score so highest-scoring moves are searched first
 - **Helper functions** :
 - `order_moves(moves, chess_obj)` : orders all moves by capture/promotion score; quiet moves get score = 0
 - `oco(moves, chess_obj)` : same scoring but filters out any non-capture and non-promotion moves; used by quiescence to only extend tactical lines

Output (as per the autograder) :

```

Playing Task3Agent vs RationalAgent.: 100%|██████████| 100/100 [05:16<00:00, 3.17s/it]
=== FINAL RESULTS ===
Task3Agent:
  Wins as White: 37
  Wins as Black: 45
  Total Wins:    82
  Avg Time for each move: 0.080270 sec
RationalAgent:
  Wins as White: 0
  Wins as Black: 1
  Total Wins:    1
  Avg Time for each move: 0.136397 sec
Draws: 17
Avg Game Length (full moves): 14.0 full moves
Avg Game Length (half moves): 29.2 plies
TASK-3 score: 81
PASSED

```

Task 4 Agent :

Approach/ Idea :

The agent uses Minimax at its core and is heavily optimized to take less time when playing
The optimization techniques that are used are as follows :

- **Alpha-Beta Pruning** : This is the primary logic. It stops searching a branch of the game tree as soon as it finds a move that is worse than one it has already found. It maintains two values, alpha (the best score the maximizer can guarantee) and beta (the best score the minimizer can guarantee), and "prunes" (stops searching) any branch where $\beta \leq \alpha$
- **Quiescence Search** : This is used to tackle the "horizon effect," where a search might stop at MAX_DEPTH right before a critical capture. When the main minimax search reaches $\text{depth} == 0$, it calls `qsearch()`. This secondary search continues exploring only moves like captures and promotions to ensure the final evaluation is based on a stable position
- **Transposition Table** : Chess has many "transpositions," where the same board position can be reached through different move orders. The agent stores the evaluation for every position it searches in a dictionary (`self.transposition_table`). Before evaluating any position, it checks this table. If the position has been seen before (at a sufficient or greater depth), it reuses the stored score, saving computation
- **Move Ordering** : The efficiency of Alpha-Beta Pruning is highly dependent on searching the best moves first. The agent has a sorting function (`order_moves`) that scores all legal moves and searches the highest-scoring ones first. This logic, known as MVV-LVA (Most Valuable Victim - Least Valuable Attacker), prioritizes moves in this order :
 - Promotions: These are almost always game-changing
 - Captures: Capturing a high-value piece with a low-value one (e.g., Pawn captures Queen) is scored highest
 - Quiet Moves: Non-captures are scored based on how much they improve the piece's position according to the piece-square tables

Algorithm/ Logic :

- **Move function call** :
 - The `move()` function is the entry point. It first resets the `transposition_table` for the new turn
 - It gets all `legal_moves`. If there are 0 or 1, it returns the obvious choice
 - It calls `self.order_moves()` to sort the legal moves at the root. This is critical for finding the best move quickly
 - It initializes $\alpha = -\text{inf}$ and $\beta = \text{inf}$
 - It then iterates through each move in the `ordered_moves` :
 - It creates a `board_copy` and makes the move
 - It calls `self.minimax(board_copy, MAX_DEPTH-1, alpha, beta, ...)` to get the score for that move
 - It updates `best_score` and `best_move` based on this score
 - It updates α (if maximizing) or β (if minimizing) at this top level
 - It returns the `best_move` it found

- **The minimax function call :**

- Transposition Table Lookup :
 - First gets a board_key and checks self.transposition_table
 - If a valid entry exists (tt_depth >= depth), it performs one of three actions:
 - TT_EXACT: The exact score is known. It returns this score immediately, pruning the entire search from this node
 - TT_ALPHA: A "fail-low" cutoff was found. It updates the local alpha to this stored score
 - TT_BETA: A "fail-high" cutoff was found. It updates the local beta to this stored score
 - If beta <= alpha after this lookup, it prunes
- Terminal Node Check :
 - It checks chess_obj.game_result()
 - If it's a win, return WIN_VALUE
 - If it's a loss, return -WIN_VALUE
 - If it's a draw, it calls self._get_draw_score()
- Depth Limit Check : If depth == 0, the main search stops. It returns the result of self.qsearch() to get a stable evaluation
- Recursive Search :
 - It gets and orders moves using order_moves()
 - It iterates through the moves, recursively calling self.minimax(..., depth-1, ...) for each move
 - It updates alpha or beta based on the returned eval_score
 - If beta <= alpha at any point, it stops looping and immediately returns the current best score.
- Before returning, it stores the calculated score, depth, and a tt_flag (TT_EXACT, TT_ALPHA, or TT_BETA) in the self.transposition_table to be used by other branches

- **qsearch() : The Quiescene Search :**

- This function is called when minimax hits depth == 0
- It first checks for game-over states (win/loss/draw), just like minimax
- It gets the "stand-pat" score by calling self.eval(). This is the score of the current position if the player does nothing
- It performs a stand-pat prune. If the stand_pat_score is already good enough to cause a cutoff it returns that score without searching any captures
- It calls self.oco() (Only Captures/Promotions) to get a sorted list of only capture and promotion moves. If there are none, it returns the stand_pat_score
- It iterates through these capture moves and employs Delta Pruning :
 - It calculates the material gain from the capture
 - It checks if stand_pat_score + gain + DELTA_PRUNING_MARGIN < alpha: continue
- It recursively calls self.qsearch(..., q_depth-1, ...) for any captures that pass the prune
- It returns the final best_score from this capture-only search

Heuristics, evaluation terms and rules :

- Material Values :
 - These are the base material weights. Pawn=100, Knight=300, Bishop=350 etc.

Python

```
PIECE_VALUE = {  
    0: 100,    # Pawn  
    1: 300,    # Knight  
    2: 350,    # Bishop  
    3: 500,    # Rook  
    4: 900,    # Queen  
    5: 9000    # King (very large)  
}
```

- Piece Square tables :
 - Each piece type has a 5×4 table
 - eval() adds/subtracts the value from these tables for positional bonus :
 - For white pieces (color == 1) it uses table[r][c]
 - For black pieces it uses table[4 - r][c] and subtracts it from score
 - The final evaluation : material + piece_square_bonus

Python

```
PAWN_TABLE = [  
    [0, 0, 0, 0],  
    [5, 10, 10, 5],  
    [10, 20, 20, 10],  
    [20, 40, 40, 20],  
    [0, 0, 0, 0]  
]  
KNIGHT_TABLE = [  
    [-10, 0, 0, -10],  
    [0, 10, 10, 0],  
    [5, 20, 20, 5],  
    [5, 15, 15, 5],  
    [-10, 0, 0, -10]  
]  
BISHOP_TABLE = [  
    [0, 5, 5, 0],  
    [5, 10, 10, 5],  
    [5, 15, 15, 5],  
    [5, 10, 10, 5],  
    [0, 5, 5, 0]  
]  
ROOK_TABLE = [  
    [0, 5, 5, 0],
```

```

        [10, 15, 15, 10],
        [5, 10, 10, 5],
        [5, 10, 10, 5],
        [0, 5, 5, 0]
    ]
    QUEEN_TABLE = [
        [0, 5, 5, 0],
        [5, 10, 10, 5],
        [5, 15, 15, 5],
        [5, 10, 10, 5],
        [0, 5, 5, 0]
    ]
    KING_TABLE = [
        [20, 30, 30, 20],
        [10, 15, 15, 10],
        [0, 0, 0, 0],
        [-10, -10, -10, -10],
        [-20, -30, -30, -20]
    ]
    PIECE_SQUARE_TABLES = {
        0: PAWN_TABLE,
        1: KNIGHT_TABLE,
        2: BISHOP_TABLE,
        3: ROOK_TABLE,
        4: QUEEN_TABLE,
        5: KING_TABLE
    }
}

```

- Pawn Structure : Penalizes "doubled pawns"
- Rook Structure : Rewards Rooks for being on "open" or "semi-open" columns
- Mobility : Rewards the player whose turn it is for having more legal moves
- Terminal/Draw : Is used by minimax() and qsearch() to evaluate draws

Output (as per the autograder) :

The below given output is achieved when the task4_agent plays with the rational player against which the task3_agent runs, to run this agent I just renamed this agent to task3_agent and recorded the results (the average time it takes to run each move and the number of wins against the rational player.)


```

Playing Task3Agent vs RationalAgent.: 100%| 100/100 [12:03<00:00, 7.24s/it]
=== FINAL RESULTS ===
Task3Agent:
Wins as White: 47
Wins as Black: 44
Total Wins: 91
Avg Time for each move: 0.404742 sec
RationalAgent:
Wins as White: 0
Wins as Black: 1
Total Wins: 1
Avg Time for each move: 0.063384 sec
Draws: 8
Avg Game Length (full moves): 15.0 full moves
Avg Game Length (half moves): 30.5 plies

```

Key Experiments and Observations :

- **Depth + Quiescence matter :**
 - Agent3 with quiescence search (qsearch()) jumps way ahead of Agent2 => 82% wins vs 54%
 - Without qsearch(), Agent2 often evaluates positions too early, stopping at volatile positions with pending captures or promotions. This leads to “horizon effect” mistakes where it misses immediate material swings. Agent3 extends the search at these tactical points, stabilizing evaluations
- **Move ordering + alpha-beta pruning increase effective search depth :**
 - Agents with move ordering (2, 3, 4) consistently outperform those without
 - The efficiency of alpha-beta pruning depends heavily on move order. Better ordering => early cutoffs => fewer nodes => more depth applied where it matters.
- **Transposition Table (TT) boosts search :**
 - TT reduces wasted computation and increases effective search depth by returning stored scores when available. Combined with ordering and qsearch, this makes the engine both faster and tactically stronger
 - Agent4, using a transposition table, and has 91% wins